

Docker 50 interview questions

Master Flow

Dockerfile

↓ docker build

Docker Image

↓ docker run

Container

↓

Main Process / Application

-p HOST:CONTAINER = port mapping

-e KEY=VALUE = runtime environment

-d = background

-it = interactive terminal

Network = container communication

Volume = persistent data

Compose = multi-container configuration

PART 1 — DOCKER FUNDAMENTALS

Q1. Docker kya hai?

Answer: Docker ek containerization platform hai jo application ko uski dependencies/runtime ke saath isolated environment yani container me package aur run karne deta hai. Main benefit reproducible aur consistent environment banana hai.

Cross Question: Docker ki zarurat kyun padi?

Docker 50 interview questions

Cross Answer: Different machines par Node version, packages, libraries aur configuration different ho sakte hain. Docker in environment/dependency differences ko kaafi had tak reduce karta hai; lekin code bug, missing secret, external service, architecture ya network problem ko automatically fix nahi karta.

Practical Example:

```
Code + Dependencies → Docker Image → Container → Application
```

Q2. Docker Container kya hai?

Answer: Container Docker Image ka created/running instance hai. Iske andar application ka main process run karta hai aur isolated runtime environment milta hai.

Cross Question: Container aur application me kya difference hai?

Cross Answer: Application actual software/process hai; container us process ko run karne wala isolated runtime unit hai.

Practical Example:

```
Image → Container → Process/Application
```

Q3. Docker Image kya hai?

Answer: Image ek packaged template hai jisme application ko run karne ke liye required files, dependencies aur metadata/layers ho sakte hain. Containers isi image se create hote hain.

Cross Question: Ek image se multiple containers bana sakte hain?

Cross Answer: Haan. Same image se multiple independent containers create kiye ja sakte hain. Starting template same ho sakta hai, runtime state independent hoti hai.

Docker 50 interview questions

Practical Example:

```
myapp:1.0 → C1, C2, C3
```

Q4. Image aur Container me difference?

Answer: Image ko blueprint/package aur container ko us blueprint ka instance samjho. Image se container create hota hai; container me application process run hota hai.

Cross Question: Container delete karne se image bhi delete hoti hai?

Cross Answer: Normally nahi. Image aur container separate Docker objects hain.

Practical Example:

```
Image → Container
```

Q5. Dockerfile kya hai?

Answer: Dockerfile ek text file hai jisme Docker Image build karne ki instructions hoti hain. Ye recipe ki tarah kaam karti hai.

Cross Question: Dockerfile → Image → Container ka flow explain karo.

Cross Answer: Dockerfile instructions deta hai; docker build image banata hai; docker run image se naya container create/start karta hai.

Practical Example:

```
Dockerfile → docker build → Image → docker run → Container
```

Q6. FROM kya karta hai?

Answer: FROM Dockerfile ka base image/stage select karta hai. Example FROM node:20.

Cross Question: FROM ko simple language me kaise samjhaoge?

Docker 50 interview questions

Cross Answer: Ye batata hai ki image kis base environment se build start hogi.

Practical Example:

```
FROM node:20
```

Q7. WORKDIR kya karta hai?

Answer: WORKDIR container/image ke andar working directory set karta hai.

Subsequent relevant instructions aur runtime context isi directory ko use kar sakte hain.

Cross Question: WORKDIR aur cd me difference?

Cross Answer: WORKDIR Dockerfile me working-directory configuration deta hai; baar-baar shell me cd karne ki zarurat kam hoti hai.

Practical Example:

```
WORKDIR /app
```

Q8. COPY kya karta hai?

Answer: COPY build context se files/directories ko image ke andar copy karta hai.

Cross Question: COPY aur RUN me difference?

Cross Answer: COPY files transfer karta hai; RUN image build ke time command execute karta hai.

Practical Example:

```
COPY package*.json ./
```

Q9. RUN kya karta hai?

Docker 50 interview questions

Answer: RUN image build karte waqt command execute karta hai, jaise npm install.

Cross Question: Kya RUN container start hone par execute hota hai?

Cross Answer: Nahi. RUN build-time instruction hai; startup ke liye CMD/ENTRYPOINT use hote hain.

Practical Example:

```
RUN npm install
```

Q10. CMD kya karta hai?

Answer: CMD container start hone par default command/parameters define karta hai.

Cross Question: RUN aur CMD me difference?

Cross Answer: RUN build time par execute hota hai; CMD runtime par default startup command define karta hai.

Practical Example:

```
CMD ["node", "server.js"]
```

PART 2 — DOCKERFILE + IMAGE BUILD

Q11. EXPOSE kya hai?

Answer: EXPOSE image me expected listening port ko document karta hai. Ye host par port automatically publish nahi karta.

Cross Question: EXPOSE aur -p me difference?

Cross Answer: EXPOSE documentation hai; -p actual host-to-container port publishing karta hai.

Practical Example:

```
EXPOSE 3000
```

Q12. docker build kya karta hai?

Answer: docker build Dockerfile aur build context ke basis par Docker Image banata hai.

Cross Question: docker build -t myapp:1.0 . ko break karo.

Cross Answer: -t name/tag set karta hai; myapp:1.0 tag hai; dot current directory ko build context banata hai.

Practical Example:

```
docker build -t myapp:1.0 .
```

Q13. Build context kya hota hai?

Answer: Build context woh files/directories ka set hai jo Docker build ko available karaya jata hai; '.' current directory ko context banata hai.

Cross Question: COPY me file nahi mil rahi ho to kya check karoge?

Docker 50 interview questions

Cross Answer: Check karo file build context ke andar hai aur .dockerignore ke through exclude nahi hai.

Practical Example:

```
docker build -t myapp:1.0 .
```

Q14. Docker image layers kya hain?

Answer: Docker images layered filesystem/content se bani hoti hain. Layers reuse/cache ho sakti hain aur build ko faster bana sakti hain.

Cross Question: Layer caching ka benefit?

Cross Answer: Unchanged build steps ki cached layers reuse ho sakti hain, isliye rebuild faster ho sakta hai.

Practical Example:

```
FROM → dependencies layer → app layer
```

Q15. docker run kya karta hai?

Answer: docker run image se naya container create karke start karta hai.

Cross Question: docker run aur docker start me difference?

Cross Answer: docker run naya container create + start karta hai; docker start existing stopped container ko start karta hai.

Practical Example:

```
docker run nginx  
docker start old_container
```

PART 3 — CONTAINER RUN + COMMANDS

Q16. docker ps kya karta hai?

Answer: docker ps currently running containers dikhata hai.

Cross Question: Saare containers kaise dekhoge?

Cross Answer: docker ps -a se running, stopped aur exited containers dikhte hain.

Practical Example:

```
docker ps
docker ps -a
```

Q17. -d flag kya hai?

Answer: -d detached mode hai. Container background me run hota hai aur terminal free rehta hai.

Cross Question: Detached container ke logs kaise dekhoge?

Cross Answer: docker logs NAME; live follow ke liye docker logs -f NAME.

Practical Example:

```
docker run -d nginx
```

Q18. -it flag kya hai?

Answer: -i input open rakhta hai aur -t terminal allocate karta hai. Dono milkar interactive terminal dete hain.

Cross Question: -it kab use karoge?

Cross Answer: Ubuntu/bash/sh jaise interactive shell ke liye.

Docker 50 interview questions

Practical Example:

```
docker run -it ubuntu bash
```

Q19. --name kya karta hai?

Answer: --name container ko explicit readable name deta hai.

Cross Question: Container name mandatory hai?

Cross Answer: Nahi. Docker automatic name generate kar sakta hai. Name mainly identification ke liye useful hai.

Practical Example:

```
docker run --name myapp myapp:1.0
```

Q20. -p HOST:CONTAINER kya karta hai?

Answer: -p host port ko container port par publish/map karta hai. -p 8080:80 ka matlab host 8080 se container 80.

Cross Question: Agar Node container me 3000 par listen karta hai aur -p 5000:3000 hai to URL kya hoga?

Cross Answer: Host se normally http://localhost:5000 access karoge; traffic container ke 3000 par jayega.

Practical Example:

```
docker run -p 5000:3000 myapp
```

Docker 50 interview questions

PART 4 — NETWORKING

Q21. -e kya karta hai?

Answer: -e runtime environment variable container ke andar pass karta hai.

Cross Question: MONGO_URI ko -e se kyun pass kar sakte hain?

Cross Answer: Database configuration ko code se separate rakhne aur environment ke hisaab se value change karne ke liye.

Practical Example:

```
docker run -e PORT=5000 myapp
```

Q22. Docker Network kya hai?

Answer: Docker Network containers ko controlled communication environment deta hai.

Cross Question: Node aur Mongo ko same network par rakhne ka benefit?

Cross Answer: Node Mongo ko service/container name se reach kar sakta hai aur related containers logically connected rehte hain.

Practical Example:

```
docker network create appnet
```

Q23. Docker me localhost ka kya meaning hai?

Answer: Container ke andar localhost normally current container ko refer karta hai, doosre container ko nahi.

Cross Question: Node container se Mongo ke liye localhost kyun wrong ho sakta hai?

Docker 50 interview questions

Cross Answer: Agar Mongo doosre container me hai to localhost Node container ko target karega. Same network par mongo:27017 use karo.

Practical Example:

```
mongodb://mongo:27017
```

Q24. Container-to-container communication kaise hoti hai?

Answer: Containers ko same Docker network par connect karke service/container name aur internal port use karke communicate karaya ja sakta hai.

Cross Question: Kya database port host par publish karna mandatory hai?

Cross Answer: Nahi. Same Docker network par internal communication host port publish kiye bina ho sakti hai.

Practical Example:

```
Node → mongo:27017
```

Q25. Bridge Network kya hai?

Answer: Bridge Docker ka common networking mechanism hai jisme connected containers ek Docker-managed virtual network par communicate karte hain.

Cross Question: Custom bridge network kyun banayenge?

Cross Answer: Related containers ko group karne aur predictable container/service-name communication ke liye.

Practical Example:

```
docker network create tarun_net
```

Q26. `host.docker.internal` kya hai?

Answer: Docker Desktop me container se host machine par chal rahi service ko access karne ke liye commonly use hone wala hostname hai.

Cross Question: Kab use karoge?

Cross Answer: Jab MongoDB ya koi API host machine par chal rahi ho aur container ko us tak pahunchna ho.

Practical Example:

```
mongodb://host.docker.internal:27017
```

Q27. Docker Volume kya hai?

Answer: Volume Docker-managed persistent storage hai. Database data ko container lifecycle se alag rakhne ke liye commonly use hota hai.

Cross Question: MongoDB ke liye volume kyun important hai?

Cross Answer: Mongo ka `/data/db` data container remove/recreate hone ke baad bhi preserve karna hota hai.

Practical Example:

```
mongo_data:/data/db
```

Q28. Named Volume vs Bind Mount?

Answer: Named volume Docker-managed storage hai; bind mount host ke actual path ko container path se connect karta hai.

Cross Question: Development me bind mount kyun useful hai?

Docker 50 interview questions

Cross Answer: Host par code/assets change karne par container me reflect karne ke liye useful hai.

Practical Example:

```
./public:/app/public
```

Q29. Anonymous Volume kya hai?

Answer: Anonymous volume Docker ka volume hota hai jiska explicit user-selected name nahi hota; Docker identifier generate karta hai.

Cross Question: Reusable database storage ke liye named volume better kyun?

Cross Answer: Named volume ko identify, inspect aur reuse karna easier hota hai.

Practical Example:

```
/data/db
```

Q30. docker stop vs docker rm?

Answer: docker stop running container ko stop karta hai; docker rm container object ko remove karta hai.

Cross Question: Stop karne ke baad container wapas start ho sakta hai?

Cross Answer: Haan, docker start se existing stopped container start ho sakta hai.

Practical Example:

```
docker stop myapp  
docker start myapp
```

Docker 50 interview questions

Q31. Docker Compose kya hai?

Answer: Docker Compose YAML file ke through multi-container application ki services, ports, networks, environment aur volumes define/manage karne deta hai.

Cross Question: Compose kyun use karte hain?

Cross Answer: Bahut saari docker run commands repeat karne ke bajay complete stack ko declarative, readable aur repeatable configuration me define kar sakte hain.

Practical Example:

```
docker compose up
```

Q32. docker compose up kya karta hai?

Answer: Compose configuration me defined services ko create/start karta hai.

Cross Question: docker compose up -d kya karega?

Cross Answer: Services ko detached/background mode me start karega.

Practical Example:

```
docker compose up -d
```

Q33. docker compose up --build kya karta hai?

Answer: Required service images ko rebuild karke stack start karta hai.

Cross Question: --build kab useful hai?

Cross Answer: Dockerfile ya build context me changes ke baad fresh image build karni ho.

Practical Example:

Docker 50 interview questions

```
docker compose up --build
```

Q34. docker compose down vs down -v?

Answer: down project containers aur default network ko remove karta hai; down -v Compose-managed volumes bhi remove kar sakta hai.

Cross Question: Database ke saath down -v dangerous kyun?

Cross Answer: Volume remove hone par persistent database data delete ho sakta hai.

Practical Example:

```
docker compose down
# carefully: docker compose down -v
```

Q35. docker compose exec kya karta hai?

Answer: Already running Compose service ke container ke andar command execute karta hai.

Cross Question: exec aur compose run me difference?

Cross Answer: exec existing running container ko target karta hai; compose run service ke liye temporary container create karta hai.

Practical Example:

```
docker compose exec nodeapp sh
```

PART 6 — REGISTRY + DEBUGGING

Q36. docker compose logs kya karta hai?

Answer: Compose services ke logs dikhata hai. -f se live follow kar sakte ho.

Cross Question: Debugging me logs kyun important hain?

Cross Answer: Startup error, connection error, application exception aur runtime output quickly identify ho sakte hain.

Practical Example:

```
docker compose logs -f nodeapp
```

Q37. Docker Hub kya hai?

Answer: Docker Hub ek public container registry hai jahan images store aur download ki ja sakti hain.

Cross Question: docker pull aur docker push me difference?

Cross Answer: pull registry se image download karta hai; push local image ko registry par upload karta hai.

Practical Example:

```
docker pull nginx  
docker push username/myapp:1.0
```

Q38. docker login kya karta hai?

Answer: Registry ke saath authentication/login karta hai.

Cross Question: Push se pehle login kab required ho sakta hai?

Cross Answer: Private/authenticated registry me push karne ke liye credentials/authentication ki zarurat hoti hai.

Docker 50 interview questions

Practical Example:

```
docker login
```

Q39. docker image inspect kya karta hai?

Answer: Image ki detailed metadata/configuration information dikhata hai.

Cross Question: docker image history ka use?

Cross Answer: Image build history aur layer-related information dekhne ke liye.

Practical Example:

```
docker image inspect myapp:1.0  
docker image history myapp:1.0
```

Q40. docker logs kya karta hai?

Answer: Container ki application output/errors dikhata hai; -f live logs follow karta hai.

Cross Question: Exited container debug karne ke liye logs useful hain?

Cross Answer: Haan. docker ps -a se exited container identify karke docker logs NAME se failure ka reason mil sakta hai.

Practical Example:

```
docker ps -a  
docker logs myapp
```

Q41. docker exec kya karta hai?

Answer: Existing running container ke andar command execute karta hai.

Cross Question: Kya stopped container par docker exec chala sakte ho?

Cross Answer: Normally nahi; target container running hona chahiye.

Docker 50 interview questions

Practical Example:

```
docker exec -it myapp sh
```

Q42. docker inspect kya karta hai?

Answer: Container ki detailed configuration/metadata jaise network, mounts, ports etc. dekhne deta hai.

Cross Question: Logs aur inspect me difference?

Cross Answer: Logs application output/errors dete hain; inspect Docker configuration/runtime metadata deta hai.

Practical Example:

```
docker inspect myapp
```

Q43. docker stats kya karta hai?

Answer: Containers ka live CPU, memory aur other resource usage dikhata hai.

Cross Question: Exit 137 par stats kyun check karoge?

Cross Answer: 137 often SIGKILL context me dikhta hai aur memory/OOM kill ek common possibility hai; resource usage investigate karna useful hai.

Practical Example:

```
docker stats
```

Q44. Container Exited kyun hota hai?

Answer: Container ka lifecycle main process se tied hota hai. Main process exit kare to container generally stop ho jata hai.

Cross Question: Exit code 0 aur 137 ka basic meaning?

Docker 50 interview questions

Cross Answer: 0 generally successful completion; 137 often SIGKILL context, commonly memory/OOM possibility ke saath investigate kiya jata hai.

Practical Example:

```
docker ps -a  
docker logs NAME
```

Q45. Docker portability kya hai?

Answer: Docker application ke runtime/dependencies ko package karke environment differences ko reduce karta hai, isliye portability improve hoti hai.

Cross Question: Kya Docker 100% guarantee deta hai ki app har machine par chalegi?

Cross Answer: Nahi. Architecture, secrets, external services, ports, network/firewall, permissions aur code bugs still problem create kar sakte hain.

Practical Example:

```
Image + compatible Docker/runtime + config + dependencies
```

PART 7 — ARCHITECTURE + SCENARIO QUESTIONS

Q46. Docker vs Virtual Machine?

Answer: Container generally lightweight aur fast-starting hota hai; VM normally full guest OS ke saath aati hai aur OS/machine-level isolation provide karti hai.

Cross Question: VM kab prefer kar sakte ho?

Cross Answer: Full guest OS, legacy OS requirement ya stronger VM-level isolation ki need ho sakti hai.

Practical Example:

VM: Host → Hypervisor → Guest OS → App

Docker: Host/Kernel → Engine → Container → App

Q47. Containerization kya hai?

Answer: Application aur dependencies ko isolated runtime unit me package karna containerization hai.

Cross Question: Virtualization se basic difference?

Cross Answer: Containers generally host kernel facilities share karte hain, jabki VM complete guest OS virtualize karti hai.

Q48. Docker Registry kya hai?

Answer: Registry container images ko store aur distribute karne ka system hai; public ya private ho sakta hai.

Cross Question: Docker Hub aur registry me relation?

Cross Answer: Docker Hub ek popular container registry service hai; registry generic concept hai.

Docker 50 interview questions

Q49. docker system df kya karta hai?

Answer: Docker disk usage ka overview deta hai, jaise images, containers aur volumes.

Cross Question: Disk space problem me ye pehle kyun useful hai?

Cross Answer: Ye batata hai Docker storage kis category me kitna space use kar raha hai, cleanup decision easier hota hai.

Practical Example:

```
docker system df
```

Q50. docker system prune kya karta hai?

Answer: Unused Docker resources ko cleanup karta hai, command/options ke scope ke according.

Cross Question: Prune carefully kyun karna chahiye?

Cross Answer: Cleanup unwanted resources remove kar sakta hai; isliye production ya important data ke saath pehle verify karna chahiye.

Practical Example:

```
docker system prune
```

FINAL INTERVIEW RAPID REVISION

Interview me sabse important cross-question pairs:

- Image vs Container
- Dockerfile vs Image
- docker build vs docker run
- docker run vs docker start
- docker stop vs docker rm
- EXPOSE vs -p
- -d vs -it
- localhost vs service/container name
- Named Volume vs Bind Mount
- docker compose up vs up -d vs up --build
- docker compose down vs down -v
- docker exec vs docker run
- docker logs vs docker inspect
- docker pull vs docker push
- Docker vs VM
- Exit 0 vs Exit 137

Most Important Commands

```
docker images
```

```
docker ps
```

Docker 50 interview questions

```
docker ps -a
```

```
docker pull IMAGE
```

```
docker build -t NAME:TAG .
```

```
docker run -d --name NAME -p HOST:CONTAINER IMAGE
```

```
docker logs NAME
```

```
docker logs -f NAME
```

```
docker exec -it NAME sh
```

```
docker inspect NAME
```

```
docker stats
```

```
docker stop NAME
```

```
docker start NAME
```

```
docker restart NAME
```

```
docker rm NAME
```

```
docker rmi IMAGE
```

```
docker network ls
```

```
docker network create NET
```

```
docker network inspect NET
```

```
docker volume ls
```

```
docker volume inspect VOLUME
```

```
docker compose up
```

```
docker compose up -d
```

```
docker compose up --build
```

```
docker compose down
```

```
docker compose down -v
```